Atualizado em 16 de out. de 2026
Criado por: Vitor Antônio Pucci Neto

Desenvolvedor Back-End Node [T6]

Mini-Projeto Avaliativo - Módulo 1 - Semana 07

SUMÁRIO

1. CONTEXTUALIZAÇÃO 1
2. DESAFIO 2
3. RESULTADOS ESPERADOS (ENTREGA) 2
    3.1. Código-fonte 2
    3.2. Repositório no GitHub 3
    3.3. Arquivo README.md 3
    3.4. Demonstração do funcionamento 3
    3.5. Submissão 4
4. REQUISITOS DAS TAREFAS 4
    4.1. Funcionalidades Obrigatórias 4
        A. Cadastro e gerenciamento de produtos 4
        B. Cadastro e consulta de clientes 5
        C. Gerenciamento do carrinho de compras 5
        D. Finalização de pedidos 6
    4.2. Requisitos Técnicos Mínimos 6
    4.3. O que não é obrigatório 7
5. SPRINTS 7
6. CRITÉRIOS DE AVALIAÇÃO 8

1. CONTEXTUALIZAÇÃO

Uma pequena loja virtual deseja organizar suas operações comerciais por meio de um sistema que permita cadastrar produtos, controlar o estoque, montar carrinhos de compras, registrar pedidos e acompanhar os resultados das vendas.

Atualmente, a loja precisa de uma solução simples que ajude a manter as informações organizadas e reduza erros comuns, como vender produtos sem estoque suficiente, calcular valores incorretamente ou registrar pedidos incompletos.

Neste projeto, você será responsável por desenvolver uma aplicação de gerenciamento de e-commerce utilizando TypeScript e Node.js, aplicando os conceitos estudados durante o módulo.

O sistema deverá trabalhar com produtos, clientes, carrinhos e pedidos. Para isso, será necessário organizar os dados por meio de tipos, criar funções reutilizáveis, implementar regras de negócio e produzir relatórios simples. O projeto não exige a criação de páginas web, integração com meios de pagamento ou conexão com banco de dados. O foco está na

construção da lógica de uma aplicação back-end, na organização do código e na aplicação correta dos conceitos da linguagem. Ao longo das tarefas, você deverá desenvolver funcionalidades que trabalhem em conjunto. Um produto cadastrado no catálogo, por exemplo, deverá poder ser adicionado ao carrinho, utilizado na criação de um pedido e contabilizado nos relatórios de vendas.

O objetivo não é criar um sistema comercial completo, mas demonstrar que você consegue organizar e desenvolver as principais funcionalidades de uma aplicação de e-commerce.

2. DESAFIO

Desenvolver um **Sistema de Gerenciamento de E-commerce com TypeScript**, capaz de administrar um catálogo de produtos, gerenciar compras e registrar pedidos, mantendo as informações consistentes durante a execução. A aplicação deverá ser organizada em arquivos e módulos, utilizar tipagem explícita nos principais dados e disponibilizar funções responsáveis pelas operações do sistema.

3. RESULTADOS ESPERADOS (ENTREGA)

Ao final do projeto, cada estudante ou squad deverá disponibilizar os seguintes itens.

3.1. Código-fonte

Um projeto TypeScript funcional, com arquivos organizados, funções reutilizáveis, regras de negócio implementadas e código compilável. Sugestão de organização:

```
miniprojeto-ecommerce/
├── src/
│   ├── types.ts
│   ├── produtos.ts
│   ├── clientes.ts
│   ├── carrinho.ts
│   ├── pedidos.ts
│   ├── relatorios.ts
│   └── main.ts
├── dist/
├── README.md
├── tsconfig.json
└── .gitignore
```

A estrutura é uma sugestão. Outros formatos serão aceitos, desde que o código esteja organizado e seja possível identificar as responsabilidades de cada arquivo.

Descrição dos arquivos sugeridos:

- types.ts: definição dos tipos utilizados pelo sistema.
- produtos.ts: cadastro, consulta, atualização e remoção de produtos.
- clientes.ts: cadastro e consulta de clientes.
- carrinho.ts: operações do carrinho e cálculos dos valores.
- pedidos.ts: finalização e consulta de pedidos.
- relatorios.ts: funções de consolidação e apresentação dos resultados.
- main.ts: inicialização da aplicação e demonstração das funcionalidades.

Os dados iniciais poderão ser definidos nos próprios arquivos responsáveis por cada módulo.

3.2. Repositório no GitHub

Criar um repositório público com o nome: Miniprojeto_Ecommerce_NomeAluno_Turma.
O repositório deverá conter o código-fonte, **o README**, o arquivo de configuração do TypeScript e os demais arquivos necessários para executar o projeto.

O nome do estudante e a identificação da turma deverão constar no README.

O repositório deverá apresentar commits com mensagens claras, registrando a evolução do trabalho. O envio poderá ser realizado pela interface do GitHub ou pelo Git, conforme as orientações trabalhadas em aula.

3.3. Arquivo README.md

O README deverá documentar:

- Nome do projeto.
- Nome do estudante e turma.
- Objetivo da aplicação.
- Funcionalidades implementadas.
- Tecnologias utilizadas.
- Estrutura dos arquivos.
- Instruções de compilação e execução.
- Exemplos das principais operações.
- Regras de negócio implementadas.
- Limitações conhecidas do sistema.

O documento deverá permitir que outra pessoa compreenda a proposta e consiga executar o projeto seguindo as instruções fornecidas.

3.4. Demonstração do funcionamento

O arquivo main.ts deverá demonstrar as principais funcionalidades da aplicação, incluindo:

- Consulta e cadastro de produtos.
- Consulta e cadastro de clientes.
- Adição e alteração de itens no carrinho.
- Cálculo dos valores da compra.
- Finalização de pelo menos um pedido.
- Atualização do estoque após a compra.
- Apresentação dos relatórios.

Também deverão ser demonstrados testes de pelo menos três situações inválidas, como tentar cadastrar um produto com preço negativo, adicionar quantidade superior ao estoque ou finalizar uma compra sem itens.

3.5. Submissão

Os **links** deverão ser submetidos na tarefa **Módulo 1 - Mini-Projeto Avaliativo**, presente na semana 07 do AVA até o dia **26/10/2026 às 22h.**

4. REQUISITOS DAS TAREFAS

4.1. Funcionalidades Obrigatórias

A. Cadastro e gerenciamento de produtos

O sistema deverá permitir:

- Cadastrar novos produtos.
- Listar todos os produtos disponíveis.
- Buscar um produto pelo seu identificador.
- Buscar produtos pelo nome, permitindo localizar parte do nome informado.
- Filtrar produtos por categoria.
- Atualizar informações de um produto cadastrado.
- Remover um produto, respeitando as regras definidas pelo sistema.

Cada produto deverá possuír no mínimo:

- id: identificador único.
- nome: nome do produto.
- categoria: categoria à qual pertence.
- preco: preço unitário.
- estoque: quantidade disponível.

Regras obrigatórias:

- O identificador de cada produto deve ser único.
- O nome não pode estar vazio.
- O preço deve ser maior que zero.
- O estoque não pode ser negativo.
- Não deve ser possível cadastrar um produto com um identificador já utilizado.

B. Cadastro e consulta de clientes

O sistema deverá trabalhar com clientes para associar os pedidos às pessoas que realizaram as compras.
Cada cliente deverá possuir:

- id: identificador único.
- nome: nome completo.
- email: endereço de e-mail.

Funcionalidades obrigatórias:

- Cadastrar clientes.
- Listar os clientes cadastrados.
- Buscar um cliente pelo identificador.
- Impedir o cadastro de clientes com identificadores ou e-mails já utilizados.
- Validar se o nome e o e-mail foram informados.

O sistema deverá iniciar com pelo menos **3 clientes cadastrados** para facilitar os testes das funcionalidades.

Não é necessário implementar autenticação, senhas ou recuperação de conta.

C. Gerenciamento do carrinho de compras

O carrinho deverá permitir que um cliente selecione os produtos que deseja comprar.

Funcionalidades obrigatórias:

- Adicionar um produto ao carrinho.
- Remover um produto do carrinho.
- Alterar a quantidade de um produto no carrinho.
- Listar os itens do carrinho.
- Calcular o subtotal de cada item.
- Calcular o valor total da compra.

Cada item do carrinho deverá conter, no mínimo, o identificador do produto e a quantidade desejada.

Regras obrigatórias:

- Não permitir quantidades iguais ou menores que zero.
- Não permitir adicionar produtos inexistentes.
- Não permitir adicionar ao carrinho uma quantidade superior ao estoque disponível.
- Se o mesmo produto for adicionado novamente, o sistema deverá atualizar sua quantidade no carrinho, em vez de criar linhas duplicadas.
- O subtotal deverá ser calculado multiplicando o preço unitário pela quantidade.
- O total do carrinho deverá corresponder à soma dos subtotais.

Exemplo: se o cliente adicionar 2 unidades de um produto que custa R$ 50,00, o subtotal desse item será R$ 100,00.

D. Finalização de pedidos

O sistema deverá permitir transformar um carrinho em um pedido registrado.

Cada pedido deverá possuir, no mínimo:

- id: identificador único.
- clienteId: identificador do cliente responsável pela compra.
- itens: produtos e quantidades compradas.
- total: valor total do pedido.
- data: data de criação do pedido.

Cada item do carrinho deverá conter, no mínimo, o identificador do produto e a quantidade desejada.

Funcionalidades obrigatórias:

- Finalizar uma compra para um cliente cadastrado.
- Registrar os itens comprados e suas quantidades.
- Calcular o valor total do pedido.
- Consultar todos os pedidos registrados.
- Consultar os pedidos de um cliente específico.

Regras obrigatórias:

- Não permitir finalizar um pedido sem itens.
- Não permitir criar pedidos para clientes inexistentes.
- Reduzir o estoque dos produtos após a criação bem-sucedida do pedido.
- Não registrar o pedido se algum item não possuir estoque suficiente.
- Gerar um identificador único para cada pedido.
- Guardar no pedido o preço praticado no momento da compra, para que alterações futuras no catálogo não modifiquem o valor de pedidos anteriores.

A finalização deverá ser consistente: se houver um problema com qualquer item, o sistema não poderá registrar parcialmente a compra nem descontar apenas parte do estoque.
Não é necessário implementar pagamento real, frete, cupons de desconto ou cálculo de impostos.

4.2. Requisitos Técnicos Mínimos

O projeto deverá atender aos seguintes requisitos:

- Utilizar TypeScript para implementar a aplicação.
- Utilizar tipos explícitos para representar produtos, clientes, itens do carrinho e pedidos.
- Criar funções com parâmetros e retornos adequadamente tipados.
- Utilizar arrays para armazenar e manipular os dados durante a execução.
- Utilizar condicionais para implementar validações e regras de negócio.

- Utilizar métodos de array, como `find()`, `filter()`, `map()`, `reduce()` ou outros adequados às funcionalidades.
- Organizar o código em arquivos e módulos, utilizando `export` e `import`.
- Compilar os arquivos TypeScript para JavaScript.
- Executar a aplicação compilada com Node.js.
- Apresentar comentários quando forem necessários para esclarecer regras ou decisões do código.

Os dados poderão permanecer em memória durante a execução. Não é obrigatório utilizar banco de dados, arquivos JSON para persistência, bibliotecas externas ou frameworks.

A aplicação deverá demonstrar o funcionamento das funcionalidades por meio de chamadas de funções e saídas compreensíveis no terminal.

Atenção: as funções não devem existir apenas para cumprir um requisito. Elas deverão ser chamadas e produzir resultados verificáveis.

4.3. O que não é obrigatório

Os itens abaixo não fazem parte dos requisitos mínimos:

- Interface gráfica ou páginas HTML.
- API REST com Express.
- Banco de dados.
- Login e autenticação.
- Integração com pagamentos.
- Controle de entregas.
- Aplicação de descontos ou cupons.
- Uso de classes e orientação a objetos, caso esses recursos não sejam necessários para a solução adotada.

Esses recursos não substituem nenhuma funcionalidade obrigatória.

5. SPRINTS

As sprints abaixo organizam o desenvolvimento do projeto em etapas. Os estudantes serão responsáveis por planejar, implementar e testar as funcionalidades, utilizando os materiais de apoio e os conceitos trabalhados durante o módulo. O professor estará disponível para suporte durante a execução.

Ao concluir cada sprint, execute o código, verifique os resultados e registre o progresso no GitHub por meio de commits com mensagens claras.

- **Aula 01 (19/10) - Configuração e gerenciamento de produtos:**
    - **Sprint 1 (Configuração e estrutura do projeto):** criar o repositório no GitHub, configurar o ambiente TypeScript e organizar os arquivos do projeto. Definir os tipos necessários para representar produtos, clientes, itens do carrinho e

pedidos. Preparar os dados iniciais de produtos e clientes e garantir que a aplicação compile e execute corretamente no Node.js.

- **Sprint 2 (Cadastro e gerenciamento de produtos):** implementar as funcionalidades de cadastro, listagem, busca por identificador, busca por nome, filtro por categoria, atualização e remoção de produtos. Aplicar as validações obrigatórias, garantindo identificadores únicos, nomes preenchidos, preços maiores que zero e estoque não negativo. Testar as operações com os dados iniciais e verificar o comportamento diante de informações inválidas.

- **Aula 02 (20/10) - Clientes, carrinho e pedidos**

  - **Sprint 3 (Cadastro de clientes e carrinho de compras):** implementar o cadastro, a listagem e a busca de clientes, aplicando as validações previstas. Desenvolver as operações de adicionar, remover e alterar itens do carrinho. Calcular os subtotais e o valor total da compra, impedindo quantidades inválidas, produtos inexistentes e solicitações superiores ao estoque disponível.
  - **Sprint 4 (Finalização e consulta de pedidos):** implementar a finalização das compras, registrando o cliente, os itens, as quantidades, os preços praticados no momento da compra, o valor total e a data do pedido. Validar a existência do cliente, a presença de itens e a disponibilidade de estoque antes de concluir a operação. Gerar identificadores únicos, atualizar o estoque somente após a validação de todos os itens e implementar a consulta de pedidos gerais e por cliente. Testar situações de sucesso e de falha.

- **Aula 03 (23/10) - Integração, revisão e entrega**

  - **Sprint 5 (Integração e testes das funcionalidades):** integrar os módulos desenvolvidos, garantindo que o cadastro de produtos e clientes, o carrinho e os pedidos funcionem em conjunto. Criar ou completar o arquivo principal (main.ts) para demonstrar as funcionalidades. Executar um fluxo completo de compra, verificar os cálculos, conferir a atualização do estoque e testar pelo menos três situações inválidas. Corrigir os problemas encontrados.
  - **Sprint 6 (Documentação e entrega):** revisar os requisitos obrigatórios, compilar e executar o projeto do início ao fim, conferir a organização dos arquivos e finalizar o README.md com objetivo, funcionalidades, tecnologias, estrutura do projeto, regras de negócio e instruções de execução. Revisar o repositório, garantir que os commits registrem o progresso realizado e enviar o link do GitHub na tarefa correspondente do AVA, dentro do prazo estabelecido.

6. CRITÉRIOS DE AVALIAÇÃO

A nota varia de **0 a 10** e o projeto tem **peso de 25% na avaliação do módulo**. Documentação soma 2,00 pontos e desenvolvimento soma 8,00 pontos. Os critérios abaixo correspondem às tarefas obrigatórias.

Projetos com plágio de soluções da internet ou de colegas receberão nota zero. É permitido consultar referências e exemplos das aulas, desde que as fontes sejam citadas e o estudante consiga explicar o código e as decisões.

Documentação do Projeto (2,00)

Nº	Critério de Avaliação	0	1,00
1	**Organização e versionamento**	Não disponibilizou uma entrega acessível ou faltam arquivos essenciais para executar o projeto.	Repositório público organizado, com código-fonte, configuração e commits que registram o progresso.
2	**Documentação e conclusões**	Não apresentou README ou o documento não permite compreender e executar o projeto.	README com identificação, objetivo, funcionalidades, estrutura, instruções de execução, regras de negócio e limitações.

Desenvolvimento do Projeto (8,00)

Nº	Critério de Avaliação	0	1,00	2,00
3	**Estrutura e tipagem**	Não apresentou estrutura funcional ou não utilizou adequadamente TypeScript.	Apresentou estrutura parcialmente funcional, com tipos ou módulos incompletos.	Organizou os módulos e utilizou tipos adequados para representar as principais entidades do sistema.
4	**Catálogo e clientes**	Não implementou as operações essenciais de cadastro e consulta.	Implementou parte das operações, mas há funcionalidades ou validações incompletas.	Implementou cadastros, consultas, alterações previstas e validações de produtos e clientes.
5	**Carrinho e cálculos**	Não implementou o carrinho ou os cálculos não funcionam corretamente.	Implementou parte das operações, com limitações nas quantidades, nos subtotais ou no total.	Implementou inclusão, alteração e remoção de itens, com validações e cálculos corretos.
6	**Pedidos e controle de estoque**	Não implementou a finalização de pedidos ou não controla o estoque corretamente.	Registra pedidos, mas apresenta falhas nas validações, nos valores históricos ou na atualização do estoque.	Registra pedidos corretamente, preserva os preços da compra, valida o estoque e evita registros parciais em caso de falha.